

Documentación Completa — Proyecto CuBirds (Tercera Entrega)

Para alguien que debe modificar el código sin conocerlo: Lee esta guía de arriba a abajo. Cada sección explica qué hace una parte, cómo funcionan los datos, y dónde cambiar si necesitas modificar algo.

ÍNDICE (Pulsa Ctrl+F para buscar)

1. **¿QUÉ ES EL JUEGO CUBIRDS?** — Entender el juego primero
 2. **ESTRUCTURA DEL PROYECTO** — Dónde están los archivos
 3. **CÓMO FUNCIONA: La Baraja de Cartas** — `DeckOfCards.java`
 4. **CÓMO FUNCIONA: Las Cartas Individuales** — `Card.java` y `TypeBird.java`
 5. **CÓMO FUNCIONA: La Mesa (Tablero)** — `Table.java`
 6. **CÓMO FUNCIONA: El Jugador** — `Player.java`
 7. **CÓMO FUNCIONA: Las Cartas Descartadas** — `DiscardedCards.java`
 8. **CÓMO FUNCIONA: El Controlador del Juego** — `Game.java` (el más importante)
 9. **CÓMO FUNCIONA: La Interfaz de Usuario** — `IU.java`
 10. **GUÍA: Cómo Modificar el Código** — Ejemplos prácticos
 11. **GUÍA: Compilar y Ejecutar** — Paso a paso
 12. **DIAGRAMA: Flujo de un Turno** — Visualización
-

¿QUÉ ES CUBIRDS?

CuBirds es un **juego de cartas para 2-5 jugadores** donde:

Objetivo

- Ser el **primero en conseguir 7 especies diferentes** de pájaros en tu zona de juego (colección).
- Si se acaban las cartas antes de que alguien gane, **gana quién tiene más cartas** en su colección.

Cómo Jugar (Resumen)

1. **Preparación:** Se reparten 8 cartas a cada jugador y se coloca la mesa con 4 filas de 3 cartas cada una.
2. **Turno de jugador:**
 - Elige una especie de pájaro que tenga en la mano
 - Elige una fila de la mesa
 - Elige colocar sus cartas a la izquierda o derecha
 - Si rodea cartas, ¡las captura y añade a su mano!
 - Opcionalmente, puede bajar especies a su colección si tiene suficientes cartas
3. **Fin de turno:** El siguiente jugador juega
4. **Fin de ronda:** Si un jugador se queda sin cartas, todos los demás se descartan, se barajan los descartes, y se reparten nuevas manos.

Regla de "Bandadas"

- Cada especie tiene un número **mínimo pequeño** (ej: 2 cartas = bandada pequeña)
- Para bajar una especie a tu colección, necesitas al menos **el número de bandada pequeña**
- Las cartas de esa especie se descartan cuando las bajas

ESTRUCTURA DEL PROYECTO

```

Proyecto/
├── pom.xml                (configuración Maven)
├── explained.md           (esta documentación)
├── src/main/java/
│   ├── es/uvigo/.../tads/ (Estructuras de Datos: List, Queue, Stack)
│   └── gal/uvigo/.../cubirds/
│       ├── core/         (LÓGICA DEL JUEGO)
│       │   ├── Game.java ★ ARCHIVO PRINCIPAL (controla todo)
│       │   ├── Player.java (datos del jugador)
│       │   ├── Table.java  (el tablero con 4 filas)
│       │   ├── DeckOfCards.java (la baraja)
│       │   ├── DiscardedCards.java (cartas descartadas)
│       │   ├── Card.java   (definición de cartas)
│       │   └── TypeBird.java (especies de pájaros)
│       └── iu/             (INTERFAZ CON EL USUARIO)
│           ├── Main.java  (punto de entrada: main())
│           └── IU.java    (lectura de teclado y mensajes)

```

🔑 Lo Más Importante

Game.java es el **corazón del proyecto**. Todos los demás archivos son herramientas que **Game** utiliza.

- **DeckOfCards, Card, Table, Player, DiscardedCards** = **modelos de datos**
- **IU** = **comunicación con el jugador**
- **Main** = **punto de inicio**

LA BARAJA DE CARTAS

Archivo: **DeckOfCards.java**

¿Qué es?

Una lista de **todas las cartas del juego, mezcladas aleatoriamente**. Funciona como:

- Quitar cartas del principio (cuando los jugadores necesitan cartas)
- Añadir cartas al final (cuando se devuelven cartas)
- Mezclar (shuffle) para que sea aleatoria

Métodos Clave

```
DeckOfCards()           // Constructor: crea todas las cartas y las mezcla
Card takeFirstCard()    // Saca la primera carta (¡se elimina!)
void addLast(Card c)     // Añade una carta al final
boolean isEmpty()       // ¿Está vacía?
int size()              // ¿Cuántas cartas hay?
void shuffle()          // Mezcla la baraja
```

Ejemplo de Modificación

Si quieres **DUPLICAR** el número de cartas en el juego:

1. Abre `DeckOfCards.java`
2. En el constructor, donde dice:

```
for (int i = 0; i < Card.values().length; i++) {
    deckOfCards.addLast(Card.values()[i]);
}
```

3. **Cambio:** Añade las cartas DOS veces:

```
for (int i = 0; i < Card.values().length; i++) {
    deckOfCards.addLast(Card.values()[i]);
}
// ← AÑADE ESTO DEBAJO
for (int i = 0; i < Card.values().length; i++) {
    deckOfCards.addLast(Card.values()[i]); // ¡Otra copia!
}
```

LAS CARTAS INDIVIDUALES

Card.java

Concepto: Cada carta es un pájaro específico (FLAMENCO_1, TUCAN_2, etc.)

```
Card.FLAMENCO_1         // Es una carta de flamenco
Card.TUCAN_5            // Es una carta de tucán
Card.values()           // Array con TODAS las cartas
```

Cada carta tiene:

- `TypeBird` — qué especie es (FLAMENCO, TUCAN, etc.)
- `smallFlock` — el mínimo de cartas para "bajar" a colección
- `largeFlock` — número más grande (no se usa en esta versión)

Métodos públicos:

```
TypeBird getTypeBird()    // ¿Qué pájaro es?  
int getSmallFlock()       // ¿Cuántas cartas necesitas para bajar?  
String toString()         // Representación: "FLAMENCO" o similar
```

TypeBird.java

Concepto: Los 8 tipos de pájaros diferentes

```
public enum TypeBird {  
    FLAMENCO, TUCAN, PATO, URRACA, PETIRROJO, LECHUZA, CURRUCA, GUACAMAYO  
}
```

Es una enumeración (enum) — un conjunto fijo de valores que no cambian.

¿Cómo se usa?

```
TypeBird pajarito = TypeBird.FLAMENCO; // Una especie  
for (TypeBird t : TypeBird.values()) { // Iterar todas las especies  
    System.out.println(t);  
}
```

LA MESA (TABLERO)

Archivo: `Table.java`

¿Qué es?

La **mesa tiene 4 filas**. Cada fila es una lista de cartas boca arriba. Los jugadores juegan sobre estas filas.

```
Mesa:  
Fila 1: [FLAMENCO] [TUCAN] [PATO]  
Fila 2: [URRACA] [PETIRROJO] [LECHUZA]  
Fila 3: [CURRUCA] [GUACAMAYO] [FLAMENCO]  
Fila 4: [TUCAN] [PATO] [URRACA]
```

Métodos Clave

```
void inicializarMesa(DeckOfCards deck)  
    // Coloca 3 cartas en cada fila al empezar.  
    // Importante: NO permite cartas del mismo tipo en la misma fila
```

```
List<Card> placeCardsOnRow(List<Card> cards, int rowIndex, boolean placeLeft)
    // Coloca cartas en una fila a la izquierda o derecha
    // DEVUELVE las cartas capturadas

void ensureRowHasTwoSpecies(int rowIndex, DeckOfCards deck, DiscardedCards disc)
    // Si una fila quedó con una sola especie, rellena con cartas del mazo
```

Regla de Captura (MUY IMPORTANTE)

Cuando un jugador coloca sus cartas, puede **rodrear** cartas de otros jugadores y capturarlas.

Ejemplo visual:

```
ANTES:  [A] [B] [B] [C]
Jugador coloca: 2x[B] a la izquierda (placeLeft=true)
DESPUÉS: [B] [B] [A] [B] [B] [C]
CAPTURADAS: [A] ← La carta entre los dos grupos de [B]
```

El código que lo hace: En `Table.placeCardsOnRow()`, busca:

1. La primera/última carta del mismo tipo que está jugando
2. Captura todo lo que esté entre las cartas jugadas y esa carta del mismo tipo

EL JUGADOR

Archivo: `Player.java`

¿Qué tiene cada jugador?

```
String name;                // Su nombre (ej: "Juan")
List<List<Card>> hand;       // Sus cartas EN LA MANO (agrupadas por tipo)
int[] speciesCounters;      // Especies en su COLECCIÓN (zona de juego)
```

Importante: La Mano está ORGANIZADA

La mano no es una lista plana de cartas. Es una **lista de listas**:

```
Mano de Juan:
├ [FLAMENCO_1, FLAMENCO_2] ← grupo 0
├ [TUCAN_1, TUCAN_2, TUCAN_3] ← grupo 1
└ [PATO_1]                  ← grupo 2
```

Así es fácil obtener "todas las cartas de FLAMENCO" de una vez.

Métodos Clave

```
void addCardToHand(Card c)
    // Añade UNA carta a la mano (la agrupa con su especie)

List<Card> takeCardsOfSpecies(TypeBird species)
    // QUITA TODAS las cartas de una especie y las devuelve
    // (La mano se queda sin esa especie)

List<TypeBird> getPlayableSpecies()
    // Devuelve qué especies puede jugar (las que tiene en la mano)

int getHandCountForSpecies(TypeBird species)
    // ¿Cuántas cartas de FLAMENCO tienes? → 2

void incrementSpeciesCounter(TypeBird species)
    // +1 a tu colección de FLAMENCO

boolean hasNoCards()
    // ¿Tu mano está vacía?
```

CARTAS DESCARTADAS

Archivo: `DiscardedCards.java`

¿Qué es?

Un **montón de cartas descartadas**. Cuando un jugador baja una especie a su colección, sus cartas van aquí.

Métodos

```
void addCard(Card c) // Añade UNA carta
void addCards(List<Card> cards) // Añade VARIAS cartas
void moveAllToDeck(DeckOfCards deck)
    // IMPORTANTE: Mueve todas las cartas aquí de vuelta a la baraja
    // (Esto ocurre cuando se agota la baraja en mitad de una ronda)
```

Flujo:

1. Jugador baja FLAMENCO a su colección → cartas van a `DiscardedCards`
2. Baraja se agota → `moveAllToDeck()` devuelve todos los descartes a la baraja
3. Se baraja todo y se reparten nuevas cartas

EL JUEGO (Game)

Archivo: `Game.java` ★

Este es el ARCHIVO MÁS IMPORTANTE

Game controla TODO. Es como el director de orquesta. Usa todos los demás para hacer funcionar el juego.

Atributos (Lo que tiene Game)

```

IU iu;                // Para hablar con el jugador
DeckOfCards deck;     // La baraja
Table table;          // El tablero
Player[] players;     // Array de jugadores (2-5)
DiscardedCards discardedCards; // Montón de descartes

```

El Flujo Principal (Método `play()`)

```

play()
├ inicializarJugadores()      → Pregunta nombres
├ repartirCartas()           → Cada jugador recibe 8 cartas
├ mostrarEstadoInicial()     → Muestra mesa + manos
└ BUCLE (mientras el juego no termine):
    ├── executePlayerTurn(currentPlayer) → Turno del jugador actual
    ├── ¿Ha ganado? (7 especies) → SÍ: termina
    ├── ¿Está sin cartas? → handleEmptyHand()
    │   ├── Los otros se descartan, repartir nuevas manos
    │   └── ¿No hay suficientes cartas? → Gana quién más tiene en colección
    └─ Siguiente jugador

```

Métodos Clave (Lee cada uno)

`executePlayerTurn(Player player)`

Un jugador hace su turno:

1. Muestra su mano y la mesa
2. Elige especie de pájaro
3. Elige fila
4. Elige lado (izquierda/derecha)
5. Coloca cartas en la mesa
6. Recibe cartas capturadas
7. Si la fila quedó con 1 especie → se rellena
8. Opcionalmente baja una especie a colección

`handleCollectionChoice(Player player)`

Pregunta si quiere bajar una especie a su colección. Si:

- Tiene suficientes cartas (ej: 2 FLAMENCOS)
- Sus cartas van al montón de descartes

- +1 a su contador de FLAMENCO

handleEmptyHand(Player currentPlayer)

Cuando un jugador se queda sin cartas:

1. Todos los demás se descartan completamente
2. Los descartes van a la baraja
3. Se baraja todo
4. Se reparten 8 cartas a cada uno
5. Si no hay suficientes cartas → FIN DEL JUEGO (gana quién más tiene)

INTERFAZ DE USUARIO

Archivo: IU.java

¿Qué es?

Maneja **toda la comunicación con el jugador por consola.**

Métodos Principales

```
void displayMessage(String msg)
    // Imprime un mensaje en pantalla

int readNumber(String msg)
    // Pregunta un número y lo devuelve
    // Valida que sea un número (no letras)

String readString(String msg)
    // Pregunta un texto

TypeBird chooseBirdType(List<TypeBird> available)
    // Muestra un menú de especies disponibles y devuelve la elegida

int chooseRow(int rowCount)
    // Muestra un menú de filas (1-4) y devuelve la elegida

boolean chooseSide()
    // Pregunta: ¿Izquierda (true) o Derecha (false)?

boolean chooseYesNo(String question)
    // Pregunta SÍ/NO
```

Ejemplo de uso en Game:

```
TypeBird chosen = iu.chooseBirdType(playable);
// Si playable tiene [FLAMENCO, TUCAN]
```



```
// Muestra: 1. FLAMENCO    2. TUCAN  
// El jugador elige 1 o 2  
// Devuelve: FLAMENCO o TUCAN
```

GUÍA PARA MODIFICAR

Cambio 1: AUMENTAR EL NÚMERO DE CARTAS INICIALES

Situación: Quieres que cada jugador empiece con 10 cartas en lugar de 8.

Archivos a cambiar: `Game.java`

Paso a paso:

1. Abre `Game.java`
2. Busca el método `repartirCartas()`:

```
private void repartirCartas() {  
    iu.displayMessage("Repartiendo cartas...");  
    dealCardsToAllPlayers(8); // ← AQUÍ está el 8  
    iu.displayMessage("Reparto completado. Cada jugador tiene 8 cartas.");  
}
```

3. **Cambio:**

```
dealCardsToAllPlayers(10); // De 8 a 10  
iu.displayMessage("Reparto completado. Cada jugador tiene 10 cartas.");
```

Cambio 2: CAMBIAR EL NÚMERO DE FILAS DE LA MESA

Situación: Quieres que la mesa tenga 6 filas en lugar de 4.

Archivos a cambiar: `Table.java`

Paso a paso:

1. Abre `Table.java`
2. Busca el constructor:

```
public Table() {  
    this.filas = new LinkedList[4]; // ← AQUÍ está el 4  
  
    for (int i = 0; i < 4; i++) { // ← Y AQUÍ también  
        this.filas[i] = new LinkedList<>();  
    }  
}
```

```
}
}
```

3. Cambio:

```
public Table() {
    this.filas = new LinkedList[6]; // 4 → 6

    for (int i = 0; i < 6; i++) { // 4 → 6
        this.filas[i] = new LinkedList<>();
    }
}
```

Cambio 3: AÑADIR UN NUEVO TIPO DE PÁJARO

Situación: Quieres añadir un nuevo pájaro: "AGUILA"

Archivos a cambiar:

1. `TypeBird.java` (añadir el enum)
2. `Card.java` (añadir cartas del nuevo tipo)
3. `Game.java` (actualizar referencias si es necesario)

Paso a paso:

Paso 1: `TypeBird.java`

```
public enum TypeBird {
    FLAMENCO, TUCAN, PATO, URRACA, PETIRROJO, LECHUZA, CURRUCA, GUACAMAYO,
    AGUILA // ← AQUÍ AÑADES
}
```

Paso 2: `Card.java` — Añade cartas al final (antes del cierre):

```
AGUILA_1(8, 12, TypeBird.AGUILA),
AGUILA_2(8, 12, TypeBird.AGUILA),
AGUILA_3(8, 12, TypeBird.AGUILA),
// ... tantas como quieras
AGUILA_10(8, 12, TypeBird.AGUILA);
```

(El 8 es el mínimo de cartas para bajar, el 12 es el máximo)

Cambio 4: CAMBIAR CUÁNDO ALGUIEN GANA

Situación: En lugar de ganar con 7 especies, quieres ganar con 5.

Archivos a cambiar: `Game.java`

Paso a paso:

1. Abre `Game.java`
2. Busca en el método `play()`:

```
if (currentPlayer.getCollectedSpeciesCount() >= 7) { // ← AQUÍ
    iu.displayMessage(currentPlayer.getName() + " Ha ganado!");
    gameFinished = true;
}
```

3. Cambio:

```
if (currentPlayer.getCollectedSpeciesCount() >= 5) { // 7 → 5
```

COMPILAR Y EJECUTAR

Desde Línea de Comandos (PowerShell en Windows)

```
# 1. Ve a la carpeta del proyecto
cd "c:\Users\Alastor\Desktop\UVIGO\PROGRAMAS PROII y
AEDII\CubirdsProject\Proyecto"

# 2. Limpia proyectos anteriores
mvn clean

# 3. Compila
mvn compile

# 4. Ejecuta
mvn exec:java -Dexec.mainClass="gal.uvigo.es.ei.aed1.cubirds.iu.Main"
```

Desde un IDE (Eclipse, IntelliJ, NetBeans)

1. Importa el proyecto como "Maven Project"
2. Espera a que descargue las dependencias
3. Busca `Main.java` en la carpeta `iu`
4. Click derecho → "Run As" → "Java Application"

Si hay Errores

Error: "Cannot find symbol" → Limpiar: `mvn clean compile`

Error: "Package not found" → Las TADs no se importan bien. Verifica que `pom.xml` tenga las rutas correctas.



DIAGRAMA DEL FLUJO DE UN TURNO

```

INICIO DEL TURNO DEL JUGADOR
    ↓
[Mostrar mano + mesa]
    ↓
[Elegir especie] ← getPlayableSpecies() + chooseBirdType()
    ↓
[Elegir fila] ← chooseRow()
    ↓
[Elegir lado] ← chooseSide()
    ↓
[Sacar cartas de mano] ← takeCardsOfSpecies()
    ↓
[Colocar en mesa] ← placeCardsOnRow()
    ↓
[Recibir cartas capturadas] ← addCardsToHand()
    ↓
[¿Fila con 1 especie?] → SÍ: ensureRowHasTwoSpecies()
    ↓
[Mostrar estado actualizado]
    ↓
[¿Quiere bajar especie?] → SÍ: handleCollectionChoice()
    ↓
FIN DEL TURNO
  
```



NOTAS FINALES

Qué Está Implementado

☒ Lógica completa del juego
 ☒ Captura de cartas correcta
 ☒ Sistema de colección (zona de juego)
 ☒ Reciclado de descartes cuando se acaba la baraja
 ☒ Determinación de ganador

Qué NO Está en Esta Versión

☒ Interfaz gráfica (es por consola)
 ☒ Guardado de partidas
 ☒ Historial de movimientos
 ☒ IA (jugadores automáticos)

Estructura de Datos Utilizada

El proyecto usa una **implementación personalizada de List** (tipo genérico, como ArrayList). Está en: [es/uvigo/esei/aed1/tads/list/LinkedList.java](https://github.com/uvigo/esei/aed1/tads/list/LinkedList.java)

Tiene métodos básicos: `add()`, `remove()`, `get()`, `size()`, etc.

Fin de la Documentación. ¡Ahora modifica con confianza! 🚀

4. DiscardedCards

- Descripción: acumulador de cartas descartadas. Implementa:
 - `void addCard(Card card)` — añade una carta al montón.
 - `void addCards(List<Card> cards)` — añade varias.
 - `boolean isEmpty()` — true si no hay descartes.
 - `int size()` — tamaño del montón.
 - `void moveAllToDeck(DeckOfCards deck)` — pasa todas las cartas descartadas al mazo y baraja el mazo.

5. Table

- Descripción: representa las 4 filas del tablero y operaciones sobre las filas.
- Atributos principales: `List<Card>[] filas` (4 listas).
- Métodos públicos y comportamiento:
 - `Table()` — constructor que crea las 4 listas vacías.
 - `void inicializarMesa(DeckOfCards deck)` — coloca 3 cartas por fila: extrae cartas del mazo y, si la carta causa duplicado de especie dentro de la fila, devuelve la carta al final del mazo hasta completar 3 especies distintas.
 - `int getRowCount()` — devuelve 4.
 - `List<Card> placeCardsOnRow(List<Card> cardsToPlay, int rowIndex, boolean placeLeft)` — coloca las cartas `cardsToPlay` en la fila `rowIndex`, en el lado izquierdo si `placeLeft==true` o derecho si es `false`.
 - Comportamiento: inserta las cartas (sin necesidad de invertir la lista de entrada). Busca si hay cartas del mismo tipo en la fila; si existe, captura las cartas entre las posiciones correspondientes y las devuelve en una lista `capturedCards`.
 - Devuelve `List<Card>` con las cartas capturadas; si no hay capturas, devuelve lista vacía.
 - `void ensureRowHasTwoSpecies(int rowIndex, DeckOfCards deck, DiscardedCards discardedCards)` — cuando una fila queda con todas cartas de la misma especie, toma cartas del mazo hasta introducir otra especie. Si el mazo se vacía, vuelca descartes al mazo y continúa.
 - `String toString()` — representación para mostrar la mesa por consola.

6. Player

- Descripción: modelo de jugador. Atributos públicos/claves: `String name`, `List<List<Card>> hand` (mano agrupada por especie), `int[] speciesCounters` (zona de juego: contador de cartas por especie).
- Métodos públicos:
 - `Player(String name)` — constructor.
 - `String getName()`.
 - `int getHandSize()` — total de cartas en la mano.
 - `int getCollectedSpeciesCount()` — número de especies distintas que el jugador ha bajado (contador > 0).
 - `int getCollectionSize()` — número total de cartas en la colección (sumatorio de counters).
 - `void incrementSpeciesCounter(TypeBird species)` — incrementa en 1 el contador de la especie en la zona de juego.
 - `List<TypeBird> getPlayableSpecies()` — devuelve lista de especies que el jugador puede jugar (una por cada grupo en la mano). Nota: los grupos vacíos no deberían existir; por eso no se comprueba `isEmpty()` en la implementación actual.

- `int getHandCountForSpecies(TypeBird species)` — número de cartas de esa especie en mano.
- `int getSmallFlockForSpecies(TypeBird species)` — devuelve el umbral de bandada pequeña asociado a la especie (se delega en la primera carta del grupo).
- `void addCardToHand(Card card)` / `void addCardsToHand(List<Card> cards)` — añaden cartas a la mano agrupadas por especie.
- `List<Card> takeCardsOfSpecies(TypeBird species)` — quita y devuelve el grupo completo de cartas de la mano para esa especie.
- `List<Card> takeAllCards()` — vacía por completo la mano devolviendo todas las cartas (usado cuando un jugador se queda sin cartas y los demás deben descartarse).
- `boolean hasNoCards()` — true si la mano está vacía.
- `String toString()` — representación de la mano para mostrar por consola.

7. Game

- Descripción: controla el flujo completo de la partida.
- Atributos principales: `IU iu`, `DeckOfCards deck`, `Table table`, `Player[] players`, `DiscardedCards discardedCards`.
- Métodos relevantes:
 - `Game(IU iu)` — constructor: inicializa mazo, mesa y descartes.
 - `void inicializarJugadores()` — pide por consola el número de jugadores (2-5) y sus nombres, creando `Player` para cada uno.
 - `void repartirCartas()` — reparte 8 cartas a cada jugador.
 - `void mostrarEstadoInicial()` — imprime mesa y manos.
 - `TypeBird elegirTipo(Player player)` — obtiene especies jugables y pide elección (IU).
 - `int elegirFila()` — pide fila al usuario (`IU.chooseRow`) — ahora muestra `Elige una fila (1-4):`.
 - `boolean elegirLado()` — pide lado (izquierda/derecha).
 - `void executePlayerTurn(Player player)` — ejecuta todo el turno: mostrar mano/mesa, elegir especie, colocar cartas, añadir cartas capturadas a la mano, rellenar fila si procede, ofrecer opción de bajar especie a zona de juego, etc.
 - `void play()` — bucle principal del juego: inicializa jugadores, reparte, ciclo de turnos, detecta victoria o fin de reparto por falta de cartas.
 - `boolean handleEmptyHand(Player currentPlayer)` — si `currentPlayer` se queda sin cartas, fuerza a los otros a descartarse (añadiendo sus cartas a `discardedCards`), vuelca descartes en `deck` y reparte; devuelve `true` si no es posible repartir y la partida debe finalizar.
 - `Player getWinnerByCollection()` — determina ganador por mayor colección en caso de final por falta de reparto.

8. IU (Interfaz de Usuario)

- Descripción: utilidades de lectura y display por consola usando `Scanner`.
- Métodos relevantes:
 - `int readNumber(String msg)` — lee un entero; reintenta hasta entrada válida.
 - `String readString(String msg)` — lee una línea.
 - `void displayMessage(String msg)` — imprime por consola.
 - `TypeBird chooseBirdType(List<TypeBird> availableTypes)` — muestra lista numerada y devuelve elección.

- `int chooseRow(int rowCount)` — muestra filas disponibles y pide número; devuelve índice 0-based. Mensaje ahora `Elige una fila (1-4):`.
- `boolean chooseSide()` — pide 1 izquierda, 2 derecha.
- `boolean chooseYesNo(String message)` — pide 1 si, 2 no.

Compilación y Ejecución — Guía Completa

Requisitos del sistema

- **Java 17+** (OpenJDK 17, Eclipse Adoptium, u otra distribución)
- **Maven 3.6+** (recomendado 3.9.x)
- Ubicación: desde la carpeta `Proyecto/` (donde está `pom.xml`)

Compilación paso a paso

Paso 1: Verifica Maven e instalación Java

```
# Verifica Maven
mvn -version

# Verifica Java
java -version
javac -version
```

Paso 2: Limpia (opcional pero recomendado) Esto elimina compilaciones previas y la carpeta `target/`:

```
mvn clean
```

Paso 3: Compila y empaqueta en JAR Desde la carpeta `Proyecto/`:

```
mvn package
```

Salida esperada (al final):

```
[INFO] Building jar: ...\\target\\cubirds_PrimerEntrega-1.0-SNAPSHOT.jar
[INFO] BUILD SUCCESS
```

Si quieres omitir pruebas (en caso de existir):

```
mvn package -DskipTests
```

Paso 4: Verifica la generación del JAR El archivo estará en:

```
target/cubirds_xxxx-x.x.x.jar
```

Verifica tamaño y existencia:

```
ls -l target/*.jar
```

Ejecución del JAR compilado

Opción A: Ejecución directa con java -jar (si pom.xml define Main-Class)

```
java -jar target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar
```

Opción B: Ejecución especificando la clase principal (recomendado)

```
java -cp target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar  
gal.uvigo.esei.aed1.cubirds.iu.Main
```

Opción C: Ejecutar directamente con Maven (sin generar JAR) Desde **Proyecto/**:

```
mvn exec:java -Dexec.mainClass="gal.uvigo.esei.aed1.cubirds.iu.Main"
```

Solución de problemas

Problema	Solución
<code>mvn: command not found</code>	Instala Maven o añádelo a PATH
<code>java: command not found</code>	Instala Java 17+ o añádelo a PATH
<code>BUILD FAILURE: "No compiler is provided"</code>	Asegúrate de tener JDK (no JRE) instalado
JAR no se ejecuta	Verifica que la clase <code>Main</code> existe en <code>gal.uvigo.esei.aed1.cubirds.iu.Main</code>
<code>OutOfMemoryException</code> en JAR	Incrementa memoria: <code>java -Xmx512m -cp ...</code>

Ejemplos y fragmentos de uso

Compilar con Maven desde la carpeta **Proyecto**:


```
mvn clean package
```

Ejecutar el juego (opción recomendada):

```
mvn exec:java -Dexec.mainClass="gal.uvigo.esei.aed1.cubirds.iu.Main"
```

O con el JAR:

```
java -cp target/cubirds_PrimerEntrega-1.0-SNAPSHOT.jar  
gal.uvigo.esei.aed1.cubirds.iu.Main
```

Flujo de juego en la consola (ejemplo simplificado):

1. Al iniciar: se pide número de jugadores (2-5) y nombres.
2. Se muestra la mesa (4 filas con 3 cartas) y las manos de cada jugador.
3. Turno 1: el primer jugador ve su mano y se le pide:
 - elegir especie jugable (p. ej. **1. Gaviota**)
 - elegir fila (**Elige una fila (1-4):**)
 - elegir lado (1 izquierda / 2 derecha)
4. Se colocan las cartas; si captura cartas, estas se añaden a la mano.
5. Si la fila quedó con una única especie, la fila se rellena hasta que aparezca otra especie (robando del mazo o vaciando descartes en el mazo si es necesario).
6. Tras la jugada, se pregunta si desea bajar una especie a su colección (bandada pequeña). Si lo hace y tiene suficientes cartas, se incrementa el contador y las cartas se descartan en **DiscardedCards**.
7. Se comprueba victoria y se pasa turno.

Notas de sintaxis y estilo para desarrolladores

- Paquete raíz: **gal.uvigo.esei.aed1.cubirds** — seguir convención de paquetes para nuevas clases.
- Evitar cambiar las firmas públicas de métodos sin actualizar los usos en **Game** y **Table**.
- Si se añaden tests, colocar fuentes de test en **src/test/java** y configurar **pom.xml** si hace falta librerías de test (JUnit 5 por ejemplo).

Pruebas recomendadas (unitarias/manuales)

- Tests unitarios sugeridos:
 - **DeckOfCardsTest**: mezclar, tamaño, **takeFirstCard()** comportamiento cuando está vacío (esperar control externo), **shuffle()**.
 - **TableTest**: **inicializarMesa()** (cada fila inicia con 3 cartas distintas), **placeCardsOnRow()** devuelve capturas correctas para varios escenarios, **ensureRowHasTwoSpecies()** rellena correctamente consumiendo descartes si el mazo se vacía.
 - **PlayerTest**: agrupar cartas por especie, **takeCardsOfSpecies()** elimina correctamente y actualiza grupos.

- `GameIntegrationTest`: simulación de varias rondas hasta victoria o fin de reparto.

Entrega y versión

- Fecha de la tercera entrega: 19 de mayo de 2026 (según enunciado).
- La rama `terceraEntrega` se ha fusionado sobre `main` en este repositorio para preparar la entrega final.

Ficheros relevantes con localización

- Código del juego: `Proyecto/src/main/java/gal/uvigo/esei/aed1/cubirds/core/` (clases `Card.java`, `DeckOfCards.java`, `DiscardedCards.java`, `Table.java`, `Player.java`, `Game.java`, `TypeBird.java`)
 - Interfaz: `Proyecto/src/main/java/gal/uvigo/esei/aed1/cubirds/iu/IU.java` y `Main.java`.
-